

Distributed Autonomous Systems 2025-26

Course Project

This project assignment is designed to give the opportunity to apply the theoretical concepts learned during the course. Students are allowed to get inspiration from any source they prefer, such as the tutorial papers in [1, 2].

Task 1 - Distributed Classification via Logistic Regression for Quality Control

Suppose we have a smart factory environment with N optical inspection agents that want to cooperatively determine a nonlinear classifier for a set of manufactured parts. Each agent i is equipped with $m_i \in \mathbb{N}$ points representing geometric features of the parts, with binary labels indicating whether the part is defective or not.

Task 1.1 - Distributed Optimization

1. As a preliminary task, implement the Gradient Tracking algorithm to solve a consensus optimization problem in the form

$$\min_z \sum_{i=1}^N \ell_i(z),$$

with $z \in \mathbb{R}^d$ and $\ell_i(z) : \mathbb{R}^d \rightarrow \mathbb{R}$ is a quadratic function for each agent $i \in \{1, \dots, N\}$.

2. Run a set of simulations to test the effectiveness of the implementation. Moreover, provide a set of solutions that includes different weighted graph patterns (e.g., cycle, path, star) whose weights are determined by the Metropolis-Hastings method.
3. Finally, for each simulation, plot the evolution of the cost function and of the norm of the gradient of the cost function across the iterations.

Task 1.2 - Centralized Classification

1. Generate a dataset of $\mathcal{M} \in \mathbb{N}$ points $\mathcal{D}^m \in \mathbb{R}^d$, with $m = 1, \dots, \mathcal{M}$.
2. Label each point with a binary label $p^m \in \{-1, 1\}$ for all $m \in \{1, \dots, \mathcal{M}\}$. The separating function is given in the form

$$\{x \in \mathbb{R}^d \mid w^\top \phi(x) + b = 0\},$$

with parameters $w \in \mathbb{R}^q$, $b \in \mathbb{R}$, and a nonlinear feature mapping $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^q$. Hence,

$$\begin{aligned} w^\top \phi(\mathcal{D}^m) + b &\geq 0 & \text{if } p^m = 1, \\ w^\top \phi(\mathcal{D}^m) + b &< 0 & \text{if } p^m = -1. \end{aligned}$$

Separating Function Rule: The specific dataset patterns and the corresponding feature mappings $\phi(x)$ to be used depend on your group number. Assuming a two-dimensional feature space $x = [x_1, x_2]^\top \in \mathbb{R}^2$:

- **Odd Groups:** You must generate datasets for, and test your algorithms against, *both* of the following feature mappings:
 - *Cubic:* $\phi(x) = [x_1, x_2, x_1^3]^\top$
 - *Superellipse:* $\phi(x) = [x_1, x_2, x_1^4, x_2^4]^\top$
 - **Even Groups:** You must generate datasets for, and test your algorithms against, *both* of the following feature mappings:
 - *Parabola:* $\phi(x) = [x_1, x_2, x_1^2]^\top$
 - *Hyperbola/Saddle:* $\phi(x) = [x_1, x_2, x_1 x_2]^\top$
3. Implement a (centralized) Gradient Method to minimize a Logistic Regression Function in order to classify the points. The resulting optimization problem is given by

$$\min_{w,b} \sum_{m=1}^{\mathcal{M}} \log \left(1 + \exp \left(-p^m (w^\top \phi(\mathcal{D}^m) + b) \right) \right).$$

where $(w, b) \in \mathbb{R}^q \times \mathbb{R}$ are the optimization variables.

4. Run a set of simulations to test the effectiveness of the implementation. Moreover, provide a set of solutions including, e.g., different dataset patterns. Finally, for each simulation, plot the evolution of the cost function and of the norm of the gradient of the cost function across the iterations.

Task 1.3 - Distributed Classification

1. Split (randomly) the dataset in N subsets, one for each agent $i \in \{1, \dots, N\}$.
2. Implement the *Gradient Tracking* algorithm to classify the dataset in a distributed way. Purposely, you must extend the code implemented in Task 1.1.
3. Generate a set of simulations testing different dataset sizes and patterns, showing the convergence of the distributed algorithm to a stationary point of the optimization problem.
4. Evaluate the quality of the obtained solution by computing the percentage of misclassified points with respect to the total number of points in the dataset.
5. Perform a comparison between the distributed and the centralized solution.

Data Splitting Rule: The splitting method and the exact percentage of data allocated to the random baseline depend on your group number G . Let $P = 40 + (G \bmod 3) \times 10$ be the baseline percentage. First, randomly select $P\%$ of the entire dataset and divide it equally among the N agents (this guarantees a baseline representation of all classes for everyone).

- **Odd Groups (Horizontal Feature-Biased Split):** Take the remaining $(100 - P)\%$ of the dataset, sort it according to the geometric value of the *first* feature ($[\mathcal{D}^m]_1$), divide it into N contiguous blocks, and assign one block to each agent.
- **Even Groups (Vertical Feature-Biased Split):** Take the remaining $(100 - P)\%$ of the dataset, sort it according to the geometric value of the *second* feature ($[\mathcal{D}^m]_2$), divide it into N contiguous blocks, and assign one block to each agent.

Task 2 - Aggregative Optimization and Safety for Multi-Robot Systems

Consider a fleet of N mobile robots in a two-dimensional environment. Denote the position of each robot $i \in \{1, \dots, N\}$ at time $t \in \mathbb{N}$ with $z_i^k \in \mathbb{R}^2$ and denote with $z^k \in \mathbb{R}^{2N}$ the stacked vector of all robot positions.

Task 2.1 - Problem Set-up and Aggregative Tracking Algorithm

The goal is to implement a distributed control algorithm allowing the robots to keep a cohesive formation while being close to their private target positions. This scenario can be formalized as the following aggregative optimization problem:

$$\begin{aligned} \min_{z \in \mathbb{R}^{2N}} \quad & \sum_{i=1}^N \ell_i(z_i, \sigma(z)), \\ \text{with} \quad & \sigma(z) \triangleq \frac{1}{N} \sum_{i=1}^N \phi_i(z_i). \end{aligned}$$

being $\ell_i : \mathbb{R}^2 \times \mathbb{R}^d \rightarrow \mathbb{R}$ the local cost function of each robot i and $\phi_i : \mathbb{R}^2 \rightarrow \mathbb{R}^d$ a local mapping. In this scenario, agent i is only aware of the decision variable z_i and of the functions ℓ_i and ϕ_i . The global aggregative variable $\sigma(z)$ can be interpreted as the barycenter of the team of robots, resulting in $\phi_i(z_i) = z_i$ for all $i \in \{1, \dots, N\}$.

1. Implement in a Python script, a version of the *Aggregative Tracking* algorithm designing the most suitable local cost functions ℓ_i which allow the robots to:
 - keep the formation tight;
 - move towards private targets.
2. Run a set of simulations showing how the team adapts its behaviour by selecting:
 - different tuning parameters of the cost function;
 - different target locations;
 - (optional) moving target locations.

Moreover, plot the evolution of the cost function and of the norm of the gradient of the cost function across the iterations.

3. Provide an animated visualization of the team behaviour.

Task 2.2 - ROS 2 Implementation

1. Create a ROS 2 package written in Python implementing the *Aggregative Tracking* algorithm previously designed.
2. Run the same set of simulations as in Task 2.1 using ROS 2, showing the same behaviour and providing the same animated visualization.

Note: In this task you can use RViz for the visualization.

Task 2.3 - Multi-Robot Safety Controllers

Add a safety layer to the control algorithm implemented in Task 2.1, ensuring that the robots safely navigate the environment without colliding with static or dynamic obstacles.

1. Characterize the Safe Set $\mathcal{C}$ using Control Barrier Functions (CBFs) to ensure obstacle avoidance. Consider a set of circular obstacles $\mathcal{O}$, where each obstacle $o \in \mathcal{O}$ is located at a given position $p_o \in \mathbb{R}^2$. For any robot $i \in \{1, \dots, N\}$ and obstacle $o \in \mathcal{O}$, define the CBF $V_{i,o}(z_i)$ as

$$V_{i,o}(z_i, p_o) = \|z_i - p_o\|^2 - d_{\text{safe}}^2,$$

being d_{safe} the minimum safety distance required between the robot and the center of the obstacle.

2. Implement a safety-critical controller using a Quadratic Program (QP). To maintain a strictly linear constraint with respect to the decision variable u_i^k , linearize the exact discrete-time CBF constraint using a first-order Taylor expansion, yielding $V_{i,o}(z_i^{k+1}) - V_{i,o}(z_i^k) \approx \nabla V_{i,o}(z_i^k)^\top u_i^k$. The actual control input u_i^* applied to the robot must minimally deviate from the nominal one while ensuring the linearized safety constraint for all obstacles:

$$\begin{aligned} u_i^{k,*} &= \arg \min_{u_i^k \in \mathbb{R}^2} \|u_i^k - u_i^{k,\text{nom}}\|^2, \\ \text{s.t.} \quad & -\nabla V_{i,o}(z_i^k)^\top u_i^k \leq \gamma V_{i,o}(z_i^k), \quad \forall o \in \mathcal{O}, \end{aligned}$$

where $u_i^{k,\text{nom}}$ is the nominal control input computed by the aggregative tracking algorithm at time k , $\nabla V_{i,o}(z_i^k)$ is the gradient of the CBF evaluated at the current position, and $\gamma \in (0, 1)$ determines the conservativeness of the safety layer.

3. Run a set of Python simulations showing the effectiveness of the CBF-QP filter compared to the purely nominal case, highlighting how the safety layer deflects the trajectories to prevent collisions with the obstacles.
4. Implement the safety layer designed in ROS 2 and show the same set of simulations, highlighting the effectiveness of the safety layer.

Notes

1. Each group must be composed of **3 students**.
2. Any other information and material necessary for the project development will be given during project meetings.
3. Each group (all members) must attend **2 mandatory meetings** with the tutor of the course, in which the work progress must be shown.
4. The project report must be written in $\text{\LaTeX}$ and must follow the main structure of the provided template.
5. Any email for project support must have the subject: **"[DAS2026] Group X: support request"**.
6. All the emails exchanged **must be cc-ed** to prof. Notarstefano, prof. Notarnicola, dr. Di Gregorio, and all the other group members.

IMPORTANT: Instructions for the Final Submission

1. The final submission deadline is 7 days before the exam date.
2. One member of the group must send an email with subject **"[DAS2026] Group X: Submission"**.
3. The final submission folder must contain:
 - README.md
 - report_group_XX.pdf
 - report (a folder containing the $\text{\LaTeX}$ code and a figs folder)
 - code (a folder containing the code relative to the main task)

References

- [1] Giuseppe Notarstefano, Ivano Notarnicola, and Andrea Camisa. Distributed optimization for smart cyber-physical networks. *Foundations and Trends in Systems and Control*, 7(3):253–383, 2019.
- [2] Andrea Testa, Guido Carnevale, and Giuseppe Notarstefano. A tutorial on distributed optimization for cooperative robotics: from setups and algorithms to toolboxes and research directions. *arXiv preprint arXiv:2309.04257*, 2023.